

Plataforma de Gestión de Amenities y Consorcio

Documento de diseño de producto y arquitectura

Sistema SaaS multi-tenant · Stack: .NET 8 · React · PostgreSQL · Nginx · Docker

Versión	1.0 — Mayo 2025
Modalidad	SaaS multi-tenant
Clientes objetivo	Edificios residenciales y barrios privados / countries
Stack tecnológico	.NET 8 · React · PostgreSQL · Nginx · Docker
Arquitectura	Clean Architecture con bounded contexts (DDD)
Fases planificadas	4 fases de desarrollo incremental

1. Visión general del producto

¿Qué problema resuelve?

Los consorcios y barrios privados gestionan sus espacios compartidos (amenities) de forma manual: WhatsApp, planillas físicas, pizarrones. Esto genera conflictos de reservas, falta de transparencia, reclamos sin seguimiento y administración ineficiente. Esta plataforma digitaliza completamente esa gestión.

Propuesta de valor

- Una sola app mobile para que el residente reserve cualquier amenity en menos de 30 segundos
- Panel administrativo para gestionar reservas, expensas, reclamos y comunicados
- Plataforma SaaS multi-tenant: un sistema, múltiples consorcios
- Modelo de negocio por suscripción mensual, sin entrega de código fuente

Los dos pilares de la plataforma

Gestión de Amenities	Administración de Consorcio
Reservas y disponibilidad en tiempo real Catálogo configurable de espacios Reglas y políticas de uso parametrizables Reportes de ocupación y analytics Cobro por reserva y depósitos	Expensas y liquidaciones Incidencias y reclamos con seguimiento Comunicados y circulares Proveedores y contratos Asambleas y votaciones

Núcleo compartido por ambos pilares

Módulo	Descripción
Identity	Residentes, unidades, roles y permisos de acceso
Notificaciones	Push, email y WhatsApp — compartido por todos los módulos
Billing	Expensas, cobros por reserva, MercadoPago
Multi-tenant	Aislamiento de datos por consorcio, configuración por tenant

2. Modelo SaaS y modelo de negocio

¿Cómo funciona el SaaS?

El cliente paga una cuota mensual y accede al sistema. Nunca recibe el código fuente. El día que deja de pagar, su tenant se desactiva y pierde el acceso. Si vuelve a pagar, el tenant se reactiva con todos los datos intactos.

¿Qué incluye la cuota mensual?

Incluido en la cuota	Se cotiza aparte
- Nuevas funcionalidades del sistema - Mejoras de UX y performance - Corrección de bugs - Soporte por canales digitales	- Carga inicial de datos del consorcio - Integración con sistemas propios del cliente - Capacitación presencial - Reportes a medida muy específicos

Planes sugeridos

Plan	Incluye	Perfil de cliente
Plan Básico	Solo gestión de amenities	Edificios chicos que quieren empezar
Plan Completo	Amenities + Administración de consorcio	Consortios medianos/grandes
Plan Multi	Varios edificios/barrios bajo un admin	Administradoras de múltiples propiedades

Ventaja competitiva (moat)

Un consorcio que usa la plataforma para liquidar expensas y gestionar reclamos tiene un costo de cambio muy alto. No puede simplemente cambiar de app — necesitaría migrar historial de expensas, reclamos, residentes y configuraciones. Eso genera retención natural.

Además, el modelo multi-tenant permite escalar sin costo operativo proporcional: los primeros 10 clientes y los primeros 200 corren sobre la misma infraestructura.

3. Concepto de tenant — Multi-tenancy

¿Qué es un tenant?

La plataforma es como un edificio de oficinas: vos sos el dueño. Cada empresa (consorcio) que alquila una oficina es un tenant. Tiene su propio espacio, su propia gente, sus propios datos — pero comparte la estructura del edificio (la infraestructura del sistema).

Cada consorcio o barrio privado que contrata la plataforma es un tenant con su propio tenant_id. Los tenants no se ven entre sí — sus datos están completamente aislados.

Ejemplo concreto

Tenant	Identificador	Configuración propia
Torre Madero	tenant_id: 1	12 unidades · Quincho · Pileta · Gym
Barrio Las Praderas	tenant_id: 2	80 lotes · Canchas · Club house
Edif. Rivadavia	tenant_id: 3	6 unidades · SUM · Laundry

Estrategia técnica de aislamiento de datos

Se utiliza una única base de datos PostgreSQL con todas las entidades incluyendo una columna **TenantId** en cada tabla. El aislamiento se garantiza mediante un filtro global en EF Core que se aplica automáticamente en todas las queries:

```
// Todas las entidades heredan de TenantEntity
public abstract class TenantEntity
{
    public Guid TenantId { get; private set; }
}

// El DbContext aplica el filtro en TODAS las queries
protected override void OnModelCreating(ModelBuilder builder)
{
    builder.Entity()
        .HasQueryFilter(b => b.TenantId == _tenantProvider.TenantId);
}

// ITenantProvider resuelve el TenantId desde el JWT en cada request
public interface ITenantProvider
{
    Guid TenantId { get; }
}
```

Super-admin (vos como dueño del SaaS)

Panel propio que ningún cliente ve. Permite ver todos los tenants activos, métricas de uso, estado de pagos y activar/desactivar tenants. Cuando un cliente deja de pagar, se cambia un flag — los datos no se borran, se deshabilita el acceso.

Cuando vuelve a pagar: reactivación inmediata con todos los datos intactos.

4. Actores del sistema

Actor	Interfaz	Responsabilidades
Residente	App mobile / PWA	Reserva amenities, cancela turnos, reporta incidencias, ve comunicados y su historial de expensas. Experiencia optimizada para mobile. Flujo de reserva en menos de 30 segundos.
Administrador liviano	Panel web simple	Ve las reservas del día, aprueba o rechaza, bloquea un amenity por mantenimiento. Interfaz similar a la app del residente pero con más información.
Administrador avanzado	Panel web completo	Accede a reportes de ocupación, gestiona usuarios y unidades, configura amenities y reglas, ve reclamos pendientes, emite expensas y gestiona proveedores.
Super-admin (vos)	Panel interno	Gestiona todos los tenants de la plataforma. Ve métricas globales, activa/desactiva consorcios y administra el negocio SaaS.

5. Catálogo de amenities comunes

Edificios residenciales

Amenity	Características
Quincho / SUM	Salón de usos múltiples con parrilla. Reserva por bloques (mañana/tarde/noche). Requiere depósito y aprobación.
Pileta / natatorio	Acceso libre con aforo máximo. Sin turno obligatorio. Puede tener franjas etarias.
Gimnasio	Acceso libre. Sin turno en general. Opcional: turnos en hora pico.
Laundry	Reserva de máquina específica. Turnos de duración fija.
Sala de reuniones	Espacio coworking. Turnos de 1-2 horas. Sin depósito.
Terraza / solarium	Acceso libre con aforo o turnos según el consorcio.

Barrios privados y countries (adicionales)

Amenity	Características
Cancha de pádel / tenis	Turnos de 60 minutos. Automático. Alta demanda. 1 reserva por día por unidad.
Cancha de fútbol	Turnos de 90 minutos. 1 reserva por semana por unidad.
Club house	Centro social con múltiples espacios. Cada sector con su configuración.
Salón de fiestas infantiles	Reserva completa del espacio. Alta anticipación requerida.
Playroom / ludoteca	Acceso libre con aforo o por turnos. Sin depósito.
Spa / sauna	Turnos individuales o por parejas. Depósito opcional.

6. Modelo de turnos — Scheduling parametrizable

El problema central

Cada amenity tiene una naturaleza distinta. Los turnos de pádel no son iguales a los del quincho, ni estos a los de la pileta. El sistema necesita ser suficientemente flexible para cubrir todos los casos sin que la configuración sea compleja para el administrador.

Las 3 estrategias de scheduling

Estrategia	Aplica a	Comportamiento
Fixed Slots (turnos fijos)	Pádel, Tenis, Fútbol, Laundry	El sistema genera matemáticamente los turnos a partir de la duración y el horario de apertura/cierre. No se persisten — se calculan al vuelo. Solo se guarda el Booking cuando alguien reserva.
Named Blocks (bloques con nombre)	Quincho, SUM, Salón de fiestas	El admin define bloques con nombre: 'Turno mañana 08-14h', 'Turno tarde 14-20h'. El sistema no genera nada — solo ofrece los bloques configurados. Puede haber tiempo de limpieza (buffer) entre bloques.
Capacity Open (acceso libre)	Pileta, Gimnasio, Playroom	No hay turnos fijos. El residente informa que va a concurrir. El sistema solo controla que la cantidad de asistentes registrados no supere el aforo máximo configurado.

Parámetros de configuración por amenity

Parámetro	Descripción	Ejemplo
SlotDurationMinutes	Duración de cada turno	60 para pádel, 90 para fútbol
MaxCapacity	Aforo máximo simultáneo	25 personas en la pileta
MaxBookingsPerUnit	Límite de reservas por unidad	1 por semana / 1 por mes
PeriodType	Período del límite	Daily, Weekly, Monthly
MinAdvanceHours	Anticipación mínima para reservar	1h para pádel, 48h para quincho
MaxAdvanceDays	Cuántos días antes se puede reservar	7 para pádel, 30 para quincho
RequiresApproval	Si el admin debe aprobar	Sí para quincho, No para pádel
RequiresDeposit	Si se cobra depósito	Sí para quincho, No para pádel
DepositAmount	Monto del depósito	\$5.000 configurable
BufferMinutes	Tiempo de limpieza entre turnos	30 min para quincho
IsOpen (por día)	Si el amenity está disponible ese día	Quincho solo vie-sáb-dom

7. Arquitectura técnica

Stack tecnológico

Capa	Tecnología
Backend	.NET 8 · Minimal APIs · MediatR · EF Core · ASP.NET Core Identity
Frontend	React · PWA (Progressive Web App) — acceso mobile desde el navegador
Base de datos	PostgreSQL — una DB, aislamiento por TenantId con query filters globales
Caché / Locks	Redis — disponibilidad en tiempo real y locks distribuidos para concurrencia
Tiempo real	SignalR — actualización instantánea de slots al reservar
Storage	MinIO (S3-compatible) — fotos de amenities, documentos del consorcio
Proxy / SSL	Nginx — reverse proxy, terminación SSL, archivos estáticos
Infraestructura	Docker + Docker Compose — contenedores para todos los servicios
Jobs	Hangfire — recordatorios, liberación de depósitos, reportes automáticos

Capas de Clean Architecture

Capa	Contiene	Rol
Domain	Entities · Aggregates · Domain Events · Value Objects · Interfaces de repositorio	La capa más interna. Sin dependencias externas. Aquí vive la lógica de negocio pura.
Application	Commands · Queries · Handlers (MediatR) · Validators · DTOs	Orquesta el dominio. Implementa los casos de uso. Usa CQRS con MediatR.
Infrastructure	EF Core · Repositorios · Email · Cache · Jobs · Integraciones externas	Implementa las interfaces del dominio. Única capa que toca la base de datos.
Presentation	Controllers · Endpoints · Middleware · DTOs de respuesta · Swagger	Expone la API REST. Traduce HTTP a comandos/queries y respuestas a DTOs.

Bounded Contexts (DDD)

Bounded Context	Responsabilidad
Reservations	El núcleo. Booking, Slot, AvailabilitySchedule, CancellationPolicy. La lógica más compleja.

Facilities	Catálogo de amenities: Amenity, AmenityType, UsageRules, Capacity, ScheduleConfig.
Identity	Residentes, unidades, roles, permisos. Encapsula ASP.NET Core Identity.
Billing	Charge, PaymentMethod, Invoice. Reactivo a Domain Events de Reservations.
Notifications	Consumer de eventos internos. ReservationConfirmed, Reminder, Cancelled.
Incidents	Reclamos e incidencias. Se conecta con Facilities (el amenity afectado).
Tenants	Gestión del tenant: alta, configuración, activación/desactivación (super-admin).

8. Gestión de concurrencia en reservas

El problema

Dos residentes pueden intentar reservar el mismo turno del quincho al mismo tiempo. Si no se maneja correctamente, ambos verían confirmación y habría un doble booking. Es el desafío técnico más crítico del sistema.

Solución recomendada: Redis distributed lock

Al iniciar el flujo de reserva, el sistema adquiere un lock distribuido en Redis sobre la clave **amenity_id + fecha + slot**. Ningún otro usuario puede completar una reserva para ese slot hasta que el primero confirme o el lock expire (TTL configurable).

```
// Al iniciar la reserva
var lockKey = $"booking-lock:{amenityId}:{date}:{slotStart}";
var lockAcquired = await _redis.SetAsync(lockKey, userId, TimeSpan.FromMinutes(5),
                                         When.NotExists);

if (!lockAcquired)
    throw new SlotUnavailableException("Otro usuario está completando esta reserva");

try {
    // Verificar disponibilidad y crear Booking
    await _bookingService.CreateAsync(command);
}
finally {
    // Liberar el lock siempre
    await _redis.DeleteAsync(lockKey);
}
```

Opciones ordenadas por complejidad

Estrategia	Complejidad	Cuando usarla
Optimistic concurrency (EF Core rowversion)	Baja	Suficiente para tráfico bajo. Detecta conflictos al guardar.
Redis distributed lock	Media	Recomendado. Previene conflictos antes de que ocurran.
PostgreSQL SELECT FOR UPDATE	Media	Alternativa si no se quiere usar Redis. Row-level locking en la DB.

9. Principios de UX — Mobile first

El principio fundamental

Complejidad interna alta — experiencia externa simple

El modelo de dominio tiene 12+ parámetros de configuración. El usuario nunca ve esos parámetros. La complejidad está encapsulada en el backend. Lo que ve el residente y el administrador tiene que ser radicalmente simple.

Un residente que quiere reservar el quincho tiene que poder hacerlo en menos de 30 segundos, sin tutoriales, sin que le expliquen nada.

Flujo del residente — 3 pasos

Paso 1	Elegir amenity	Lista de espacios disponibles con estado visual inmediato: verde = libre, rojo = ocupado.
Paso 2	Elegir fecha y turno	Calendario simple. Los slots ocupados se muestran en rojo, los libres en verde. Un toque selecciona. Las reglas del amenity se aplican automáticamente.
Paso 3	Confirmar	Pantalla de confirmación con resumen. Un botón. Notificación inmediata.

Simplificación para el administrador — Presets

Cuando el admin da de alta un amenity, elige el tipo desde un menú de presets. El sistema pre-configura automáticamente todos los parámetros técnicos del tipo elegido. El admin solo ajusta lo que necesita cambiar:

Preset	Configuración pre-aplicada
Cancha de pádel / tenis	FixedSlots 60min, sin depósito, sin aprobación, lun-dom
Cancha de fútbol	FixedSlots 90min, sin depósito, sin aprobación, 1/semana
Quincho / SUM	NamedBlocks, con depósito, con aprobación, vie-sáb-dom
Pileta / natatorio	CapacityOpen, sin turno, aforo máximo, sin depósito
Gimnasio	CapacityOpen, sin turno, sin aforo obligatorio
Otro espacio	Configuración personalizada desde cero

Dos niveles de usuario administrativo

Perfil	Acceso y funciones
--------	--------------------

Admin liviano (encargado del consorcio)	Ve reservas del día, aprueba/rechaza, bloquea amenity por mantenimiento. Interfaz simple, similar a la app del residente con más info.
Admin avanzado (administrador del consorcio)	Reportes de ocupación, gestión de usuarios y unidades, configuración de amenities y reglas, gestión de reclamos, emisión de expensas, proveedores.

10. Roadmap de desarrollo — 4 fases

Fase 1 — MVP

- Catálogo de amenities con configuración de scheduling (3 estrategias)
- Reservas con disponibilidad en tiempo real (SignalR)
- App mobile (PWA React) para residentes
- Panel admin básico: aprobar/rechazar reservas
- Multi-tenant: Identity, TenantId en todas las entidades
- Notificaciones push: confirmación y cancelación de reservas
- Docker Compose con todos los servicios

Fase 2 — Pagos y comunicacion

- Cobro por reserva y gestión de depósitos
- Integración con MercadoPago
- Notificaciones por WhatsApp Business API
- Comunicados y circulares del consorcio
- Cancelaciones con políticas configurables (penalidades)
- Panel admin con reportes básicos de ocupación

Fase 3 — Administracion de consorcio

- Módulo de incidencias y reclamos con seguimiento de estado
- Expensas y liquidaciones mensuales
- Gestión de proveedores y contratos
- Asambleas y votaciones digitales
- Analytics avanzado de uso de amenities
- Panel super-admin para gestión de todos los tenants

Fase 4 — Premium y escala

- Integraciones con smart locks y acceso por QR
- Integración con Google Calendar / iCal
- API pública para integraciones con sistemas de terceros
- Planes diferenciados por tenant (básico vs completo)
- Portal de onboarding self-service para nuevos consorcios
- Analytics predictivo y recomendaciones de capacidad

11. Entidades principales del dominio

Entidad: AmenityScheduleConfig

Campo	Tipo	Descripción
TenantId	Guid	Aislamiento por consorcio
AmenityId	Guid FK	Amenity al que pertenece esta configuración
SlotStrategy	Enum	FixedSlots NamedBlocks CapacityOpen
SlotDurationMinutes	int	Para FixedSlots: duración de cada turno
MaxCapacity	int	Para CapacityOpen: aforo máximo simultáneo
MaxBookingsPerUnit	int	Límite de reservas por unidad
PeriodType	Enum	Daily Weekly Monthly
MinAdvanceHours	int	Anticipación mínima para reservar
MaxAdvanceDays	int	Máximo de días hacia adelante que se puede reservar
RequiresApproval	bool	Si el admin debe aprobar cada reserva
RequiresDeposit	bool	Si se cobra depósito al reservar
DepositAmount	decimal	Monto del depósito (reembolsable)
BufferMinutes	int	Tiempo de limpieza bloqueado entre turnos

Entidad: Booking

Campo	Tipo	Descripción
TenantId	Guid	Aislamiento por consorcio
AmenityId	Guid FK	El amenity reservado
UnitId	Guid FK	La unidad que reserva (depto, lote)
ResidentId	Guid FK	El residente que hizo la reserva
Date	DateOnly	Fecha de la reserva
StartTime	TimeOnly	Hora de inicio
EndTime	TimeOnly	Hora de fin
SlotTemplateName	string	Para NamedBlocks: nombre del bloque (ej: Turno tarde)
Status	Enum	Pending Confirmed Rejected Cancelled Completed
DepositAmount	decimal	Monto de depósito cobrado (0 si no aplica)
DepositReturned	bool	Si el depósito fue devuelto

CancelledAt	DateTime?	Timestamp de cancelación si aplica
CreatedAt	DateTime	Timestamp de creación

12. Próximos pasos

Decisiones de arquitectura pendientes

Tema	Descripción
Modelo de datos del tenant	Diseñar la entidad Tenant completa, el flujo de alta y la gestión de planes (básico vs completo).
Bounded context Incidents	Diseñar el flujo de incidencias: apertura, asignación a proveedor, seguimiento, cierre y confirmación del residente.
Estructura de la solución .NET	Organización de proyectos, namespaces y carpetas siguiendo la arquitectura propia del equipo.
API Contract del MVP	Definir los endpoints REST y DTOs del MVP: autenticación, listado de amenities, disponibilidad y booking.
Panel admin liviano	Diseñar el flujo del administrador para aprobar/rechazar reservas y bloquear amenities.
Onboarding de un nuevo consorcio	Definir el flujo de alta de un nuevo tenant: registro, configuración inicial y carga de datos.

Este documento refleja el estado del diseño de producto y arquitectura al inicio del proyecto. Todos los parámetros, nombres de entidades y decisiones tecnológicas son revisables antes del inicio del desarrollo. Se recomienda actualizar este documento a medida que evolucionen las decisiones.